Министерство науки и высшего образования Российской Федерации
Московский авиационный институт
(национальный исследовательский университет)
Институт «Компьютерные науки и прикладная математика»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Теория автоматов и формальных языков»

Часть 1. Задания 1–3: регулярная грамматика, конечный автомат, минимизация ДКА

Вариант № 4.4 : распознавание списка адресов электронной почты

Студент: Гуляев А.П.
Группа: М8О-413Б-23
Руководитель: к.т.н, доцент ОЦ 8 Киндинова В. В.
Оценка: _____
Дата: _____

Москва, 2026

СОДЕРЖАНИЕ

ЗАДАНИЯ И ИСХОДНЫЕ ДАННЫЕ....3
1 Задание 1. Проектирование регулярной грамматики....4
1.1 Анализ паттерн-модели....4
1.2 Терминальный алфавит....4
1.3 Грамматика G....5
1.4 Примеры выводов....6
2 Задание 2. Конечный автомат по грамматике....8
2.1 Построение НДКА по грамматике....8
2.2 Алгоритм преобразования НДКА в ДКА (метод подмножеств)....9
2.3 Применение алгоритма....10
2.4 Полученный ДКА....13
2.5 Программная реализация....15
3 Задание 3. Минимизация (редукция) ДКА....17
3.1 Алгоритм минимизации....17
3.2 Применение алгоритма к ДКА M′....18
3.3 Минимальный ДКА M″....20
3.4 Свойства автомата и языка....21
ЗАКЛЮЧЕНИЕ....23
ПРИЛОЖЕНИЕ А: Листинг программы на Go....24
А.1 Тест EmailListDfa_test.go....27

ЗАДАНИЯ И ИСХОДНЫЕ ДАННЫЕ

Часть 1 курсового проекта включает следующие задания:

1) Спроектировать грамматику по паттерн-модели регулярного языка.
2) Преобразовать спроектированную грамматику в конечный автомат, составить диаграмму переходов КА. Если получается НДКА, то выполнить алгоритм преобразования НДКА в ДКА. Привести алгоритм преобразования НДКА в ДКА. Реализовать получившийся ДКА.
3) Изучить и применить алгоритм минимизации (редукции) ДКА.

Паттерн-модель варианта (синтаксис регулярных выражений .NET):

```
^((\w+([-+.]\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*)\s*[;,.]{0,1}\s*)+$
```

Паттерн описывает список из одного или нескольких адресов электронной почты. Каждый адрес может сопровождаться пробельными символами и не более чем одним разделителем из множества {; , .}. Разделитель необязателен, поэтому два адреса могут идти друг за другом вплотную: например, цепочка a@b.cc+d@e.f принадлежит языку (она разбивается на адреса a@b.c и c+d@e.f).

1 Задание 1. Проектирование регулярной грамматики

1.1 Анализ паттерн-модели

Якоря ^ и $ означают, что паттерн должен совпасть со всей входной цепочкой. Внешняя группа (…)+ повторяется один или более раз. Обозначим: E — адрес, W — один символ класса \w, S — один символ класса \s, Z = [;,.]. Тогда

$L = (E \cdot S^* \cdot Z? \cdot S^*)^+$,

$E = W^+ ([-+.] W^+)^* @ W^+ ([-.] W^+)^* \. W^+ ([-.] W^+)^*$.

Адрес E состоит из трёх фрагментов: локальной части $W^+([-+.]W^+)^*$ до знака @; доменной части $W^+([-.]W^+)^*$, за которой следует обязательная точка \.; завершающей части $W^+([-.]W^+)^*$ (домен верхнего уровня, возможно с дефисами и точками). Квантификатор {0,1} эквивалентен «?», т. е. разделитель Z может отсутствовать.

1.2 Терминальный алфавит

Классы \w и \s содержат много символов, но внутри каждого класса все символы ведут себя одинаково. Поэтому каждый класс заменяется одним терминалом. Терминальный алфавит грамматики:

T = { w, s, +, −, ., @, ;, , }.

Таблица 1 – Терминалы грамматики

Терминал	Что обозначает	Где используется в паттерне
w	любой символ класса \w (буква, цифра, «_»)	все части адреса
s	любой символ класса \s (пробел, табуляция и т. п.)	\s* после адреса
+	знак плюс	только в локальной части
−	дефис	локальная часть, домен
.	точка	локальная часть, домен, обязательная точка, разделитель Z
@	«собака»	граница локальной части и домена
; ,	точка с запятой, запятая	разделитель Z

Символ, не входящий ни в один класс (например, «!»), не имеет в грамматике правил; при распознавании он приводит к отказу. Точка — самый

«нагруженный» символ: она встречается в четырёх местах паттерна. Именно это, а также возможность начать новый адрес сразу после старого, делают автомат по грамматике недетерминированным.

1.3 Грамматика G

Строим праволинейную (автоматную) грамматику G = (T, V, P, S_0): каждое правило имеет вид A → aB или A → a, где a ∈ T, A, B ∈ V. Пустых правил (ε-правил) в грамматике нет; пустая цепочка языку не принадлежит.

V = { S_0, A, B, C, D, E, F, G, H, I, J }, P = { p_1, …, p_{42} }, начальный символ — S_0.

Таблица 2 – Назначение нетерминальных символов

Нетерминал	Смысл (что уже прочитано / что ожидается)	Фрагмент паттерна
S_0	начальный символ: ожидается первый символ адреса	\w+ …
A	читается слово локальной части	\w+([-+.]\w+)*
B	после разделителя «+», «−» или «.» в локальной части; ждём \w	[-+.]\w+
C	после «@»; ждём первый символ домена	@\w+
D	читается слово доменной части	\w+([-.]\w+)*
E	после «−» или «.» внутри домена; ждём \w	[-.]\w+
F	после обязательной точки; ждём первый символ последней части	\.\w+
G	читается слово последней части; адрес можно завершить	\w+([-.]\w+)*
H	после «−» или «.» в последней части; ждём \w	[-.]\w+
I	после адреса прочитаны пробельные символы	\s*
J	прочитан разделитель Z (и, возможно, пробелы после него)	[;,.]{0,1}\s*

Правила грамматики P

1. Локальная часть адреса \w+([-+.]\w+)*@:

p_1: S_0 → wA	p_2: A → wA	p_3: A → +B	p_4: A → −B
p_5: A → .B	p_6: B → wA	p_7: A → @C	

2. Доменная часть \w+([-.]\w+)* **и обязательная точка** \. (правило p_{13}):

p_8: C → wD	p_9: D → wD	p_{10}: D → −E	p_{11}: D → .E
p_{12}: E → wD	p_{13}: D → .F		

3. Последняя часть \w+([-.]\w+)*:

p_{14}: F → wG	p_{15}: F → w	p_{16}: G → wG	p_{17}: G → w

p_{18}: G → −H	p_{19}: G → .H	p_{20}: H → wG	p_{21}: H → w

4. Хвост \s*[;,.]{0,1}\s* **и переход к следующему адресу** (внешний «+»):

p_{22}: G → sI	p_{23}: G → s	p_{24}: G → ;J	p_{25}: G → ,J
p_{26}: G → .J	p_{27}: G → ;	p_{28}: G → ,	p_{29}: G → .
p_{30}: G → wA	p_{31}: I → sI	p_{32}: I → s	p_{33}: I → ;J
p_{34}: I → ,J	p_{35}: I → .J	p_{36}: I → ;	p_{37}: I → ,
p_{38}: I → .	p_{39}: I → wA	p_{40}: J → sJ	p_{41}: J → s
p_{42}: J → wA			

Правила вида X → a (p_{15}, p_{17}, p_{21}, p_{23}, p_{27}–p_{29}, p_{32}, p_{36}–p_{38}, p_{41}) завершают вывод: они соответствуют местам, где по паттерну цепочка может закончиться. Конкретно: после символа последней части адреса (F, G, H), после пробела (G, I, J) и после разделителя (G, I).

Правила p_{30}, p_{39}, p_{42} реализуют внешнее повторение (…)+: терминал w в них — первый символ локальной части следующего адреса.

1.4 Примеры выводов

Индекс у стрелки ⇒ — номер применённого правила. Терминал w в выводе заменён конкретным символом, ␣ обозначает пробел.

1) Цепочка a@b.c (один адрес):

$S_0 \Rightarrow_1 aA \Rightarrow_7 a@C \Rightarrow_8 a@bD \Rightarrow_{13} a@b.F \Rightarrow_{15} a@b.c$

2) Цепочка u.s@d.ru; (разделитель в локальной части и «;» в конце):

$S_0 \Rightarrow_1 uA \Rightarrow_5 u.B \Rightarrow_6 u.sA \Rightarrow_7 u.s@C \Rightarrow_8 u.s@dD \Rightarrow_{13} u.s@d.F$

$\Rightarrow_{14} u.s@d.rG \Rightarrow_{16} u.s@d.ruG \Rightarrow_{27} u.s@d.ru;$

3) Цепочка a@b.c,␣d@e.f (два адреса, запятая и пробел):

$S_0 \Rightarrow_1 aA \Rightarrow_7 a@C \Rightarrow_8 a@bD \Rightarrow_{13} a@b.F \Rightarrow_{14} a@b.cG \Rightarrow_{25} a@b.c,J$

$\Rightarrow_{40} a@b.c,␣J \Rightarrow_{42} a@b.c,␣dA \Rightarrow_7 a@b.c,␣d@C \Rightarrow_8 a@b.c,␣d@eD$

$\Rightarrow_{13} a@b.c,␣d@e.F \Rightarrow_{15} a@b.c,␣d@e.f$

4) Цепочка a@b.cc+d@e.f (два адреса без разделителя):

$S_0 \Rightarrow_1 aA \Rightarrow_7 a@C \Rightarrow_8 a@bD \Rightarrow_{13} a@b.F \Rightarrow_{14} a@b.cG \Rightarrow_{30} a@b.ccA$

$\Rightarrow_3$ a@b.cc+B $\Rightarrow_6$ a@b.cc+dA $\Rightarrow_7$ a@b.cc+d@C $\Rightarrow_8$ a@b.cc+d@eD

$\Rightarrow_{13}$ a@b.cc+d@e.F $\Rightarrow_{15}$ a@b.cc+d@e.f

В примере 4 по правилу p_{30} второй символ «c» стал началом нового адреса c+d@e.f.

2 Задание 2. Конечный автомат по грамматике

2.1 Построение НДКА по грамматике

Для праволинейной грамматики конечный автомат строится по правилам:

- каждому нетерминалу ставится в соответствие состояние; начальное состояние — S_0;
- правило $X \to aY$ даёт переход $X \xrightarrow{a} Y$;
- правило $X \to a$ даёт переход $X \xrightarrow{a} q_f$, где q_f — добавленное заключительное состояние.

Получаем $KA = (Q, \Sigma, \delta, S_0, F)$, $L(KA) = L(G)$, где

$Q = \{ S_0, A, B, C, D, E, F, G, H, I, J, q_f \}$ — 12 состояний;

$\Sigma = \{$ w, s, +, −, ., @, ;, , $\}$ — входной алфавит;

S_0 — начальное состояние; $F = \{ q_f \}$ — множество заключительных состояний.

Функция переходов $\delta: Q \times \Sigma \to 2^Q$ задана таблицей 3 (∅ — переход отсутствует).

Таблица 3 – Функция переходов НДКА

Состояние	**w**	**s**	**+**	**−**	**.**	**@**	**;**	**,**
→ S_0	{A}	∅	∅	∅	∅	∅	∅	∅
A	{A}	∅	{B}	{B}	{B}	{C}	∅	∅
B	{A}	∅	∅	∅	∅	∅	∅	∅
C	{D}	∅	∅	∅	∅	∅	∅	∅
D	{D}	∅	∅	{E}	**{E, F}**	∅	∅	∅
E	{D}	∅	∅	∅	∅	∅	∅	∅
F	{G, q_f}	∅	∅	∅	∅	∅	∅	∅
G	**{A, G, q_f}**	{I, q_f}	∅	{H}	**{H, J, q_f}**	∅	{J, q_f}	{J, q_f}
H	{G, q_f}	∅	∅	∅	∅	∅	∅	∅
I	{A}	{I, q_f}	∅	∅	{J, q_f}	∅	{J, q_f}	{J, q_f}
J	{A}	{J, q_f}	∅	∅	∅	∅	∅	∅
q_f *	∅	∅	∅	∅	∅	∅	∅	∅

Почему автомат недетерминированный

Из одного состояния по одному символу есть несколько переходов (выделено в таблице):

– $\delta(D, .) = \{E, F\}$: точку в домене нельзя сразу отнести ни к разделителю поддоменов (правило p_{11}), ни к обязательной точке \. (p_{13}) — это выяснится только дальше;
– $\delta(G, w) = \{A, G, q_f\}$: очередной символ \w может продолжать последнюю часть адреса (p_{16}, p_{17}) или начинать новый адрес (p_{30});
– $\delta(G, .) = \{H, J, q_f\}$: точка после адреса — это либо разделитель внутри последней части (p_{19}), либо разделитель списка Z (p_{26}, p_{29});
– правила вида $X \to aY$ и $X \to a$ с одним терминалом дают пары $\{Y, q_f\}$.

Следовательно, нужно выполнить преобразование НДКА в ДКА.

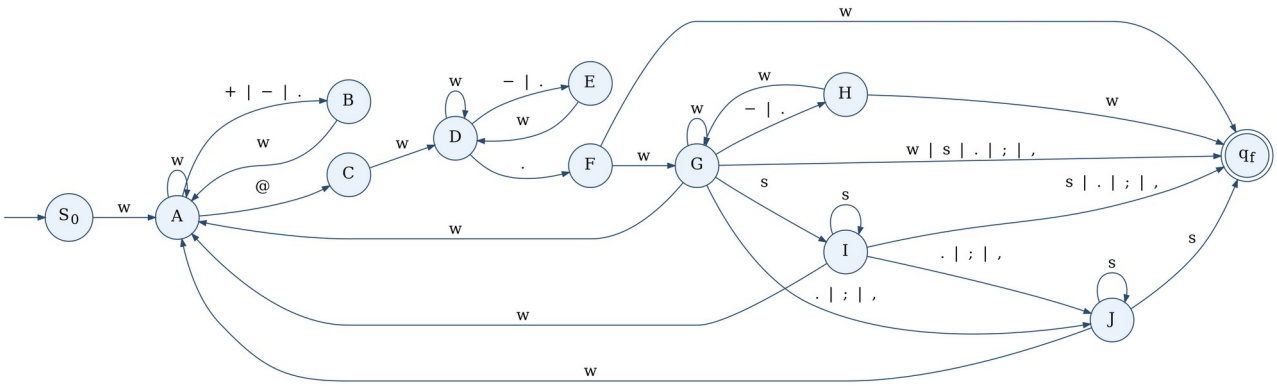


Рисунок 1 – Диаграмма переходов НДКА

2.2 Алгоритм преобразования НДКА в ДКА (метод подмножеств)

Вход: НДКА $M = (Q, \Sigma, \delta, q_0, F)$. **Выход:** ДКА $M_D = (Q_D, \Sigma, \delta_D, U_0, F_D)$, $L(M_D) = L(M)$.

Идея. Состояние ДКА — это множество тех состояний НДКА, в которых НДКА может находиться после чтения одного и того же префикса. ДКА «отслеживает все ветви НДКА одновременно».

1) Начальное состояние ДКА: $U_0 = \{q_0\}$. Поместить U_0 в Q_D как необработанное.
2) Пока в Q_D есть необработанное множество U: пометить U обработанным и для каждого символа $a \in \Sigma$ вычислить $U' = \delta_D(U, a) = \bigcup_{q \in U} \delta(q, a)$.

3) Если $U' \neq \varnothing$ и $U' \notin Q_D$, добавить U' в Q_D как необработанное. Записать переход $\delta_D(U, a) = U'$. (Если $U' = \varnothing$, переход не определён — в полном автомате он ведёт в состояние отказа $q_\perp = \varnothing$.)
4) Заключительные состояния: $F_D = \{ U \in Q_D \mid U \cap F \neq \varnothing \}$, т. е. все множества, содержащие q_f.

Строятся только достижимые подмножества, поэтому вместо $2^{12} = 4096$ возможных получается лишь несколько. Если бы в НДКА были ε-переходы, вместо $U_0 = \{q_0\}$ брали бы ε-замыкание $E(\{q_0\})$, а результат каждого шага также замыкали бы: $\delta_D(U, a) = E(\bigcup \delta(q, a))$. В нашем НДКА ε-переходов нет.

2.3 Применение алгоритма

Перечислены только непустые переходы. Жирным отмечены новые состояния.

$A_0 = \{S_0\}$

$\delta_D(A_0, w) = \delta(S_0, w) = \{A\} \Rightarrow$ **$A_1 = \{A\}$**

$A_1 = \{A\}$

$\delta_D(A_1, w) = \{A\} = A_1$

$\delta_D(A_1, +) = \delta_D(A_1, -) = \delta_D(A_1, .) = \{B\} \Rightarrow$ **$A_2 = \{B\}$**

$\delta_D(A_1, @) = \{C\} \Rightarrow$ **$A_3 = \{C\}$**

$A_2 = \{B\}$

$\delta_D(A_2, w) = \delta(B, w) = \{A\} = A_1$

$A_3 = \{C\}$

$\delta_D(A_3, w) = \delta(C, w) = \{D\} \Rightarrow$ **$A_4 = \{D\}$**

$A_4 = \{D\}$

$\delta_D(A_4, w) = \{D\} = A_4$

$\delta_D(A_4, -) = \{E\} \Rightarrow$ **$A_5 = \{E\}$**

$\delta_D(A_4, .) = \{E, F\} \Rightarrow$ **$A_6 = \{E, F\}$**

A_5 = {E}

$\delta_D(A_5, w) = \delta(E, w) = \{D\} = A_4$

A_6 = {E, F}

$\delta_D(A_6, w) = \delta(E, w) \cup \delta(F, w) = \{D\} \cup \{G, q_f\} \Rightarrow$ **A_7 = {D, G, q_f}** (заключительное)

A_7 = {D, G, q_f}

$\delta_D(A_7, w) = \{D\} \cup \{A, G, q_f\} \Rightarrow$ **A_8 = {A, D, G, q_f}** (заключительное)

$\delta_D(A_7, s) = \varnothing \cup \{I, q_f\} \Rightarrow$ **A_9 = {I, q_f}** (заключительное)

$\delta_D(A_7, -) = \{E\} \cup \{H\} \Rightarrow$ **A_{10} = {E, H}**

$\delta_D(A_7, .) = \{E, F\} \cup \{H, J, q_f\} \Rightarrow$ **A_{11} = {E, F, H, J, q_f}** (заключительное)

$\delta_D(A_7, ;) = \delta_D(A_7, ,) = \{J, q_f\} \Rightarrow$ **A_{12} = {J, q_f}** (заключительное)

A_8 = {A, D, G, q_f}

$\delta_D(A_8, w) = \{A\} \cup \{D\} \cup \{A, G, q_f\} = A_8$

$\delta_D(A_8, s) = \{I, q_f\} = A_9$

$\delta_D(A_8, +) = \delta(A, +) = \{B\} = A_2$

$\delta_D(A_8, -) = \{B\} \cup \{E\} \cup \{H\} \Rightarrow$ **A_{13} = {B, E, H}**

$\delta_D(A_8, .) = \{B\} \cup \{E, F\} \cup \{H, J, q_f\} \Rightarrow$ **A_{14} = {B, E, F, H, J, q_f}** (заключительное)

$\delta_D(A_8, @) = \delta(A, @) = \{C\} = A_3$

$\delta_D(A_8, ;) = \delta_D(A_8, ,) = \{J, q_f\} = A_{12}$

A_9 = {I, q_f}

$\delta_D(A_9, w) = \{A\} = A_1$

$\delta_D(A_9, s) = \{I, q_f\} = A_9$

$\delta_D(A_9, .) = \delta_D(A_9, ;) = \delta_D(A_9, ,) = \{J, q_f\} = A_{12}$

A_{10} = {E, H}

$\delta_D(A_{10}, w) = \{D\} \cup \{G, q_f\} = A_7$

A_{11} = {E, F, H, J, q_f}

$\delta_D(A_{11}, w) = \{D\} \cup \{G, q_f\} \cup \{G, q_f\} \cup \{A\} = A_8$

$\delta_D(A_{11}, s) = \delta(J, s) = \{J, q_f\} = A_{12}$

A_{12} = {J, q_f}

$\delta_D(A_{12}, w) = \{A\} = A_1$

$\delta_D(A_{12}, s) = \{J, q_f\} = A_{12}$

A_{13} = {B, E, H}

$\delta_D(A_{13}, w) = \{A\} \cup \{D\} \cup \{G, q_f\} = A_8$

A_{14} = {B, E, F, H, J, q_f}

$\delta_D(A_{14}, w) = \{A\} \cup \{D\} \cup \{G, q_f\} \cup \{G, q_f\} \cup \{A\} = A_8$

$\delta_D(A_{14}, s) = \{J, q_f\} = A_{12}$

Новых множеств больше не появляется — алгоритм завершён. Получено 15 состояний A_0…A_{14}. Заключительные (содержат q_f): A_7, A_8, A_9, A_{11}, A_{12}, A_{14}.

2.4 Полученный ДКА

$M' = (K', \Sigma, \delta', A_0, F')$, $K' = \{A_0, \ldots, A_{14}\}$, $F' = \{A_7, A_8, A_9, A_{11}, A_{12}, A_{14}\}$. Звёздочкой отмечены заключительные состояния, «—» — переход не определён (в полном автомате — переход в $q_\perp$).

Таблица 4 – Функция переходов ДКА M'

ДКА	Множество НДКА	w	s	+	−	.	@	;	,
→ A_0	{S_0}	A_1	—	—	—	—	—	—	—
A_1	{A}	A_1	—	A_2	A_2	A_2	A_3	—	—
A_2	{B}	A_1	—	—	—	—	—	—	—
A_3	{C}	A_4	—	—	—	—	—	—	—
A_4	{D}	A_4	—	—	A_5	A_6	—	—	—
A_5	{E}	A_4	—	—	—	—	—	—	—
A_6	{E, F}	A_7	—	—	—	—	—	—	—
A_7 *	{D, G, q_f}	A_8	A_9	—	A_{10}	A_{11}	—	A_{12}	A_{12}
A_8 *	{A, D, G, q_f}	A_8	A_9	A_2	A_{13}	A_{14}	A_3	A_{12}	A_{12}
A_9 *	{I, q_f}	A_1	A_9	—	—	A_{12}	—	A_{12}	A_{12}
A_{10}	{E, H}	A_7	—	—	—	—	—	—	—
A_{11} *	{E, F, H, J, q_f}	A_8	A_{12}	—	—	—	—	—	—
A_{12} *	{J, q_f}	A_1	A_{12}	—	—	—	—	—	—
A_{13}	{B, E, H}	A_8	—	—	—	—	—	—	—
A_{14} *	{B, E, F, H, J, q_f}	A_8	A_{12}	—	—	—	—	—	—

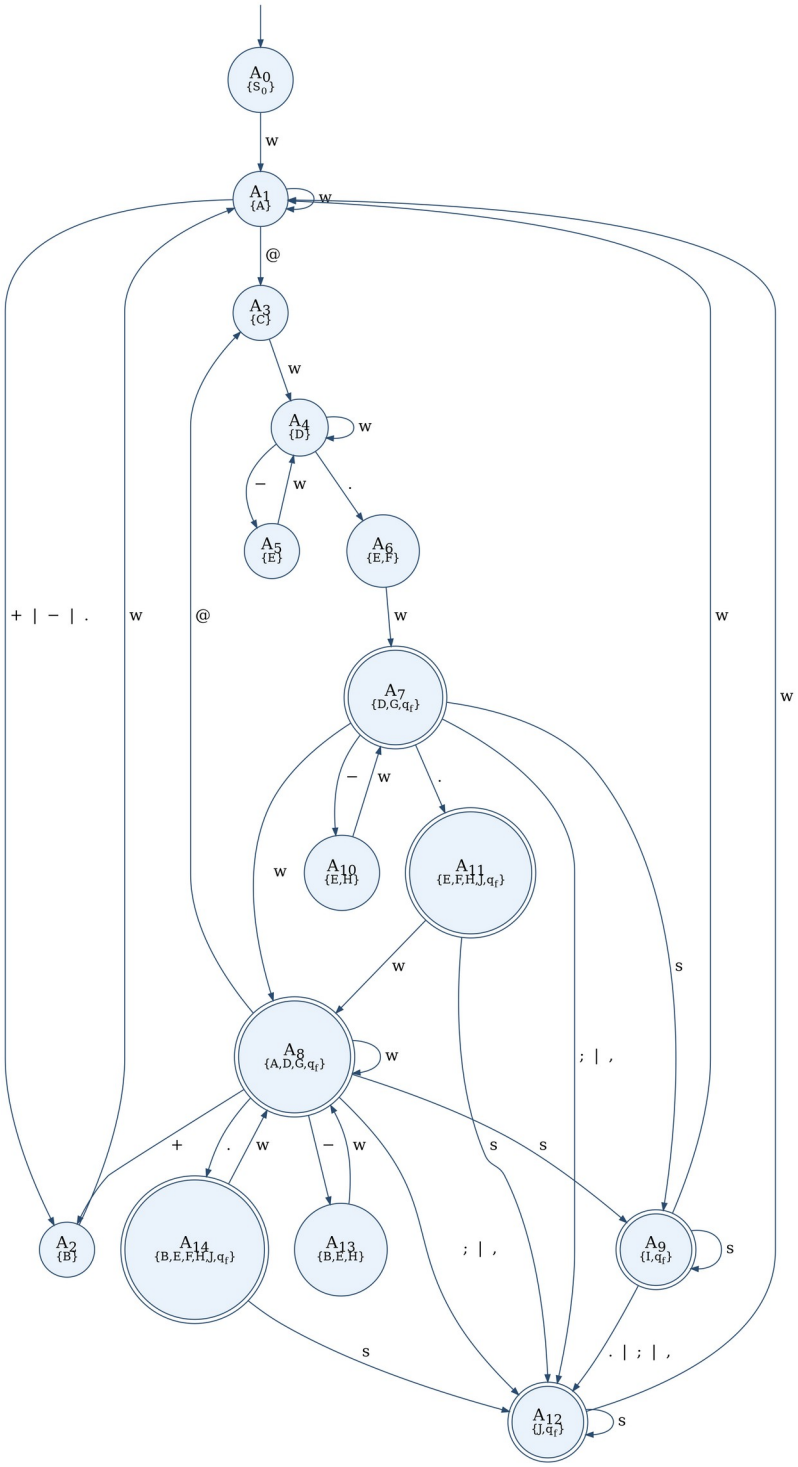


Рисунок 2 – Диаграмма переходов ДКА M′ (после метода подмножеств)

Примеры конфигураций ДКА

Конфигурация — пара (текущее состояние, непрочитанная часть входа); ⊢ — один такт работы.

1) (A_0, a@b.c) ⊢ (A_1, @b.c) ⊢ (A_3, b.c) ⊢ (A_4, .c) ⊢ (A_6, c) ⊢ (A_7, ε) — $A_7 \in F'$, принята.

2) (A_0, a@b.c,␣d@e.f) ⊢ (A_1, @b.c,␣d@e.f) ⊢ (A_3, b.c,␣d@e.f) ⊢ (A_4, .c,␣d@e.f)

⊢ (A_6, c,␣d@e.f) ⊢ (A_7, ,␣d@e.f) ⊢ (A_{12}, ␣d@e.f) ⊢ (A_{12}, d@e.f) ⊢ (A_1, @e.f)

⊢ (A_3, e.f) ⊢ (A_4, .f) ⊢ (A_6, f) ⊢ (A_7, ε) — принята.

3) (A_0, a@b.c@e.f) ⊢ … ⊢ (A_7, @e.f): $\delta'(A_7, @)$ не определён — цепочка отвергнута.

В примере 3 из $A_7 = \{D, G, q_f\}$ нет перехода по «@»: последняя часть адреса состоит из одного символа «c», поэтому он не может одновременно быть началом следующего адреса. A из $A_8 = \{A, D, G, q_f\}$ переход по «@» есть — поэтому цепочка a@b.cd@e.f принадлежит языку.

2.5 Программная реализация

Программа написана на Go (приложение А). Реализован **минимальный** ДКА из задания 3 (11 рабочих состояний и состояние отказа $q_\perp$); по построению он распознаёт тот же язык, что и M′. Устройство программы:

- функции isWord и isSpace относят входной символ к классу w или s (аналоги \w и \s в .NET: буквы, десятичные цифры, комбинируемые знаки, знаки соединения, включая «_»; пробельные символы);
- функция nextState(state, r) — это функция переходов δ: оператор switch по состоянию, затем проверка класса символа; для пары, у которой перехода нет, возвращается qDead ($q_\perp$);
- функция Run читает цепочку посимвольно (по рунам Unicode), запоминает трассу состояний и останавливается при попадании в $q_\perp$; цепочка принимается, если последнее состояние — одно из $q_6 \ldots q_{10}$;
- main читает строки со стандартного ввода, для каждой печатает трассу и результат; ввод exit завершает работу.

Регулярные выражения в программе не используются. Для проверки корректности написан тест (EmailListDfa$_{test}$.go): он перебирает **все** цепочки

длины от 0 до 7 над алфавитом представителей классов {a, ␣, +, −, ., @, ;, ,} — 2 396 745 цепочек — и сравнивает ответ автомата с ответом исходного регулярного выражения. Расхождений нет.

Пример работы программы

```
Проверка строк конечным автоматом. Для выхода введите exit.
> user@mail.com
Трасса: q0 → q1 → q1 → q1 → q1 → q3 → q4 → q4 → q4 → q4 → q5 → q6 → q7 → q7
Цепочка принадлежит языку.
> a@b.c, d@e.f
Трасса: q0 → q1 → q3 → q4 → q5 → q6 → q10 → q10 → q1 → q3 → q4 → q5 → q6
Цепочка принадлежит языку.
> a@b.cc+d@e.f
Трасса: q0 → q1 → q3 → q4 → q5 → q6 → q7 → q0 → q1 → q3 → q4 → q5 → q6
Цепочка принадлежит языку.
> a@b.c@e.f
Трасса: q0 → q1 → q3 → q4 → q5 → q6 → q⊥
Цепочка не принадлежит языку.
> user@domain
Трасса: q0 → q1 → q1 → q1 → q1 → q3 → q4 → q4 → q4 → q4 → q4 → q4
Цепочка не принадлежит языку.
> exit
```

Таблица 5 – Результаты проверки

Входная цепочка	Результат	Пояснение
user@mail.com	принята	один адрес
ivan.petrov+work@sub-domain.example.com	принята	разделители в локальной и доменной частях
user@mail.com; other@mail.com	принята	разделитель «;» и пробел
a@b.c, d@e.f	принята	разделитель «,»
user@mail.com.	принята	точка в конце — разделитель Z
user@mail.com ;	принята	пробел перед разделителем
a@b.cc+d@e.f	принята	адреса a@b.c и c+d@e.f без разделителя
иван@почта.рф	принята	кириллица входит в класс \w
a@b.c@e.f	отвергнута	«c» не может быть и концом, и началом адреса
@mail.com	отвергнута	пустая локальная часть
user@domain	отвергнута	нет обязательной точки
user@mail..com	отвергнута	после точки нужен символ \w
user@mail.com;;other@x.y	отвергнута	не более одного разделителя
a@b.c-	отвергнута	после дефиса нужен символ \w
(пустая строка)	отвергнута	внешний «+» требует хотя бы один адрес

Все ответы программы совпадают с ответами исходного регулярного выражения.

3 Задание 3. Минимизация (редукция) ДКА

3.1 Алгоритм минимизации

Вход: ДКА $M' = (K', \Sigma, \delta', H', F')$. **Выход:** ДКА M'' с наименьшим числом состояний, $L(M'') = L(M')$.

Эквивалентность состояний. Состояния p и q эквивалентны (неразличимы), если для любой цепочки $x \in \Sigma^*$ автомат из p и из q либо в обоих случаях принимает x, либо в обоих отвергает: $\delta'^*(p, x) \in F' \Leftrightarrow \delta'^*(q, x) \in F'$. Состояния k-эквивалентны ($p \equiv_k q$), если это выполняется для всех цепочек длины не больше k. Отношение эквивалентности разбивает K' на классы; каждый класс заменяется одним состоянием.

1) **Удаление недостижимых состояний.** Строим множество достижимых: $R_0 = \{H'\}$, $R_{i+1} = R_i \cup \{\delta'(q, a) \mid q \in R_i, a \in \Sigma\}$, пока множество растёт. Состояния вне R удаляются.
2) **Пополнение.** Неопределённые переходы считаются переходами в непринимающее состояние $q_\perp$. На практике это значит: если у p и q по символу a переходы не определены оба — это совпадение, если только у одного — различие.
3) **Начальное разбиение Π_0 (0-эквивалентность).** Два класса: заключительные F' и незаключительные $K' \setminus F'$. Пустая цепочка ε уже различает эти состояния.
4) **Построение Π_{k+1} по Π_k.** Для каждого состояния записываем *сигнатуру*: в какие классы Π_k ведут его переходы по каждому символу. Состояния одного класса Π_k с одинаковыми сигнатурами остаются вместе, с разными — разделяются. Формально: $p \equiv_{k+1} q \Leftrightarrow p \equiv_k q$ и $\forall a \in \Sigma: \delta'(p, a) \equiv_k \delta'(q, a)$.
5) **Останов.** Если $\Pi_{k+1} = \Pi_k$, то $\Pi = \Pi_k$ — классы эквивалентности. Число шагов не больше $|K'| - 2$.

6) **Построение M″.** Состояния — классы [q] ∈ П; начальное — класс, содержащий H′; заключительные — классы из состояний F′; δ″([q], a) = [δ′(q, a)]. Результат не зависит от выбора представителя класса, потому что разбиение устойчиво.

Сравниваются именно **классы**, а не имена состояний: переходы в разные, но эквивалентные состояния не мешают объединению.

3.2 Применение алгоритма к ДКА M′

Шаг 1. Недостижимые состояния. Все 15 состояний $A_0 \ldots A_{14}$ получены методом подмножеств из A_0, т. е. достижимы по построению. Удалять нечего.

Шаг 2. Начальное разбиение Π_0.

$G_1 = F' = \{A_7, A_8, A_9, A_{11}, A_{12}, A_{14}\}$, $G_2 = K' \setminus F' = \{A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_{10}, A_{13}\}$.

Шаг 3. Разбиение Π_1. В таблице для каждого состояния указан класс Π_0, в который ведёт переход («—» — переход не определён).

Таблица 6 – Сигнатуры состояний относительно Π_0

Сост.	w	s	+	−	.	@	;	,	Подкласс
A_7	G_1	G_1	—	G_2	G_1	—	G_1	G_1	I
A_8	G_1	G_1	G_2	G_2	G_1	G_2	G_1	G_1	II
A_9	G_2	G_1	—	—	G_1	—	G_1	G_1	III
A_{11}	G_1	G_1	—	—	—	—	—	—	IV
A_{14}	G_1	G_1	—	—	—	—	—	—	IV
A_{12}	G_2	G_1	—	—	—	—	—	—	V
A_0	G_2	—	—	—	—	—	—	—	VI
A_2	G_2	—	—	—	—	—	—	—	VI
A_3	G_2	—	—	—	—	—	—	—	VI
A_5	G_2	—	—	—	—	—	—	—	VI
A_1	G_2	—	G_2	G_2	G_2	G_2	—	—	VII
A_4	G_2	—	—	G_2	G_2	—	—	—	VIII
A_6	G_1	—	—	—	—	—	—	—	IX
A_{10}	G_1	—	—	—	—	—	—	—	IX
A_{13}	G_1	—	—	—	—	—	—	—	IX

Класс G_1 распадается на пять подклассов: $\{A_7\}$, $\{A_8\}$, $\{A_9\}$, $\{A_{11}, A_{14}\}$, $\{A_{12}\}$. Например, A_7 и A_8 различаются по «+» (у A_8 переход есть, у A_7 нет), A_9 и A_{12} —

по «.», «;», «,». Класс G_2 распадается на четыре: $\{A_0, A_2, A_3, A_5\}$, $\{A_1\}$, $\{A_4\}$, $\{A_6, A_{10}, A_{13}\}$.

$\Pi_1 = \{ \{A_0, A_2, A_3, A_5\}, \{A_1\}, \{A_4\}, \{A_6, A_{10}, A_{13}\}, \{A_7\}, \{A_8\}, \{A_9\}, \{A_{11}, A_{14}\}, \{A_{12}\} \}$ — 9 классов.

Шаг 4. Разбиение Π_2. Одноэлементные классы делить нельзя. Проверяем остальные:

- $\{A_0, A_2, A_3, A_5\}$: у всех определён только переход по w; $\delta'(A_0, w) = \delta'(A_2, w) = A_1 \in \{A_1\}$, а $\delta'(A_3, w) = \delta'(A_5, w) = A_4 \in \{A_4\}$. Классы $\{A_1\}$ и $\{A_4\}$ разные $\Rightarrow$ расщепление на $\{A_0, A_2\}$ и $\{A_3, A_5\}$.
- $\{A_6, A_{10}, A_{13}\}$: $\delta'(A_6, w) = \delta'(A_{10}, w) = A_7 \in \{A_7\}$, а $\delta'(A_{13}, w) = A_8 \in \{A_8\} \Rightarrow$ расщепление на $\{A_6, A_{10}\}$ и $\{A_{13}\}$.
- $\{A_{11}, A_{14}\}$: $\delta'(A_{11}, w) = \delta'(A_{14}, w) = A_8$, $\delta'(A_{11}, s) = \delta'(A_{14}, s) = A_{12}$, остальные не определены у обоих $\Rightarrow$ не расщепляется.

  $\Pi_2 = \{ \{A_0, A_2\}, \{A_1\}, \{A_3, A_5\}, \{A_4\}, \{A_6, A_{10}\}, \{A_7\}, \{A_8\}, \{A_9\}, \{A_{11}, A_{14}\}, \{A_{12}\}, \{A_{13}\} \}$ — 11 классов.

Шаг 5. Разбиение Π_3 (проверка останова).

- $\{A_0, A_2\}$: оба по w переходят в A_1 — один класс;
- $\{A_3, A_5\}$: оба по w переходят в A_4 — один класс;
- $\{A_6, A_{10}\}$: оба по w переходят в A_7 — один класс;
- $\{A_{11}, A_{14}\}$: по w — в A_8, по s — в A_{12} — совпадает.

Ни один класс не расщепился: $\Pi_3 = \Pi_2$. Алгоритм завершён. Число состояний сократилось с 15 до 11 (с 16 до 12 с учётом состояния отказа $q_\perp$).

Смысл объединённых состояний

- $A_0 = \{S_0\}$ и $A_2 = \{B\}$: в обоих ждём непустое слово локальной части — «начало слова до @».
- $A_3 = \{C\}$ и $A_5 = \{E\}$: ждём слово домена (после «@» или после «−» в домене).

– A_6 = {E, F} и A_{10} = {E, H}: ждём \w, после которого адрес уже можно закончить; продолжения одинаковы.
– A_{11} и A_{14} отличаются только элементом B; но из B есть лишь переход по w в A, а из A_{11} по w уже достигается A_8, содержащий A. Поэтому B ничего не добавляет.

Не объединяются, например, A_6 и A_{13}: цепочка x = w+w@w.w принимается из A_{13} (после первого w попадаем в A_8, где возможен новый адрес), но отвергается из A_6 (из A_7 нет перехода по «+»).

3.3 Минимальный ДКА M″

Классам присвоены имена $q_0 \ldots q_{11}$; имя q_2 отдано состоянию отказа $q_\perp$, чтобы номера совпадали с константами в программе (qDead = 2).

$M'' = (K'', \Sigma, \delta'', q_0, F'')$,

$K'' = \{ q_0, q_1, q_\perp, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11} \}$, $F'' = \{ q_6, q_7, q_8, q_9, q_{10} \}$.

Таблица 7 – Функция переходов полного минимального ДКА M″

Сост.	Класс	w	s	+	−	.	@	;	,
→ q_0	{A_0, A_2}	q_1	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
q_1	{A_1}	q_1	$q_\perp$	q_0	q_0	q_0	q_3	$q_\perp$	$q_\perp$
q_3	{A_3, A_5}	q_4	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
q_4	{A_4}	q_4	$q_\perp$	$q_\perp$	q_3	q_5	$q_\perp$	$q_\perp$	$q_\perp$
q_5	{A_6, A_{10}}	q_6	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
q_6 *	{A_7}	q_7	q_8	$q_\perp$	q_5	q_9	$q_\perp$	q_{10}	q_{10}
q_7 *	{A_8}	q_7	q_8	q_0	q_{11}	q_9	q_3	q_{10}	q_{10}
q_8 *	{A_9}	q_1	q_8	$q_\perp$	$q_\perp$	q_{10}	$q_\perp$	q_{10}	q_{10}
q_9 *	{A_{11}, A_{14}}	q_7	q_{10}	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
q_{10} *	{A_{12}}	q_1	q_{10}	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
q_{11}	{A_{13}}	q_7	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$
$q_\perp$	∅	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$	$q_\perp$

Состояние $q_\perp$ — поглощающее: из него по любому символу автомат остаётся в $q_\perp$. По любому символу, не входящему в Σ (например, «!»), из любого состояния автомат переходит в $q_\perp$.

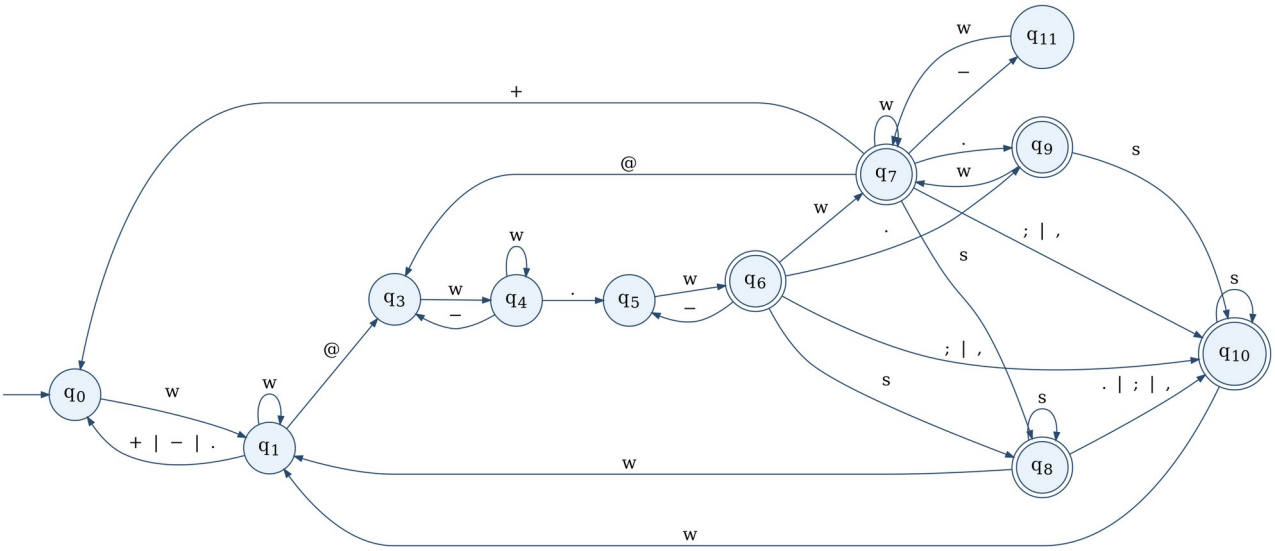


Рисунок 3 – Диаграмма переходов минимального ДКА M″ (переходы в $q_\perp$ не показаны)

Таблица 8 – Смысл состояний минимального ДКА

Состояние	Смысл	Имя в программе
q_0	ожидается первый символ слова локальной части	qStart
q_1	читается локальная часть	qLocal
q_3	после «@» или «−» в домене, ждём \w	qAfterAt
q_4	читается домен	qDomain
q_5	после точки (или «−» за ней) в домене, ждём \w	qAfterDot
q_6	адрес может быть завершён; после точки прочитан ровно один символ \w	qEmailEnd
q_7	адрес может быть завершён; последний символ \w может начать новый адрес	qEmailWord
q_8	после адреса прочитаны пробелы	qSpaces
q_9	прочитана точка: разделитель списка или часть домена	qAfterFinalDot
q_{10}	прочитан разделитель списка (и, возможно, пробелы)	qAfterSep
q_{11}	«−» в состоянии q_7: продолжение домена или локальной части нового адреса	qAfterHyphen
$q_\perp$	отказ, выхода нет	qDead

3.4 Свойства автомата и языка

– M″ детерминированный и полный: для каждого состояния и каждого символа определён ровно один переход.

– M″ минимальный: последний шаг алгоритма не расщепил ни одного класса. Минимальный полный ДКА единствен с точностью до переименования состояний.

– Пустая цепочка не принадлежит языку: начальное состояние q_0 не заключительное.

– Язык бесконечный: в автомате есть циклы (например, петля $q_4 \xrightarrow{w} q_4$), лежащие на пути из q_0 в заключительное состояние.
– Язык регулярный: он задан праволинейной грамматикой и конечным автоматом.

Лемма о накачке для регулярных языков. Если L регулярен, то существует $p \geq 1$, такое что любая цепочка $z \in L$, $|z| \geq p$, представима как $z = xyv$, где $|xy| \leq p$, $|y| \geq 1$ и $xy^iv \in L$ для всех $i \geq 0$.

Для нашего языка можно взять $p = 12$ — число состояний M″. Читая первые 12 символов цепочки, автомат проходит 13 состояний (с начальным), и по принципу Дирихле какое-то состояние повторяется. Участок y между повторами — цикл; его можно пройти 0, 1, 2, … раз, и автомат всё равно закончит в том же заключительном состоянии. Например, для z = user@mail.com: x = u, y = s (петля $q_1 \xrightarrow{w} q_1$), v = er@mail.com; цепочки uer@mail.com, ussser@mail.com также принадлежат языку.

ЗАКЛЮЧЕНИЕ

В части 1 курсового проекта выполнено следующее:

- по паттерн-модели спроектирована праволинейная грамматика G с 11 нетерминалами и 42 правилами, приведены примеры выводов;
- по грамматике построен конечный автомат; он оказался недетерминированным (причины: многозначность точки и возможность начать новый адрес без разделителя);
- методом подмножеств НДКА преобразован в ДКА из 15 состояний; приведены таблица, диаграмма и примеры конфигураций;
- алгоритмом разбиения на классы эквивалентности ДКА минимизирован до 11 рабочих состояний (12 с состоянием отказа);
- минимальный ДКА реализован на Go; корректность подтверждена полным перебором цепочек длины до 7 (2 396 745 цепочек) со сравнением с исходным регулярным выражением.

ПРИЛОЖЕНИЕ А

Листинг программы на Go

Файл EmailListDfa.go. Сборка и запуск: go run EmailListDfa.go. Тест: go test.

```
package main


import (
    "bufio"
    "fmt"
    "os"
    "unicode"
)


// Состояния минимального ДКА. Номер константы совпадает с индексом
// состояния q0…q11 в отчёте (q2 = q⊥ — поглощающее состояние отказа).
const (
    qStart         = iota // q0  = {A0, A2}:  ждём первый символ локальной части
    qLocal                // q1  = {A1}:      читаем локальную часть
    qDead                 // q⊥  (q2):        поглощающее состояние отказа
    qAfterAt              // q3  = {A3, A5}:  после @ или «-» в домене ждём \w
    qDomain               // q4  = {A4}:      читаем часть домена
    qAfterDot             // q5  = {A6, A10}: после «.» или «-» в домене ждём \w
    qEmailEnd             // q6  = {A7}:      адрес можно завершить (одна \w после точки)
    qEmailWord            // q7  = {A8}:      адрес можно завершить, либо \w начала новый
адрес
    qSpaces               // q8  = {A9}:      после адреса прочитаны пробелы
    qAfterFinalDot        // q9  = {A11, A14}: точка — разделитель домена или списка
    qAfterSep             // q10 = {A12}:     прочитан разделитель списка [;,.]
    qAfterHyphen          // q11 = {A13}:     «-» после ≥ 2 символов \w в конце домена
)


// stateNames — имена состояний для вывода трассы, как в отчёте.
var stateNames = [...]string{"q0", "q1", "q⊥", "q3", "q4", "q5", "q6", "q7", "q8", "q9",
"q10", "q11"}


// isWord приближённо воспроизводит смысл \w в исходном .NET-паттерне:
// буквы, десятичные цифры, комбинируемые знаки и знаки соединения (включая _).
func isWord(r rune) bool {
    return unicode.IsLetter(r) ||
        unicode.IsDigit(r) ||
        unicode.Is(unicode.Mn, r) ||
        unicode.Is(unicode.Pc, r)
}


// isSpace соответствует классу \s: пробельные символы.
func isSpace(r rune) bool {
    return unicode.IsSpace(r)
}


// nextState выполняет один шаг автомата по текущему состоянию и символу.
// Если для пары «состояние + символ» переход не предусмотрен, уходим в qDead.
func nextState(state int, r rune) int {
    w := isWord(r)
    s := isSpace(r)


    switch state {
    case qStart:
        if w {
            return qLocal
        }
        return qDead


    case qLocal:
        if w {
            return qLocal
```

```
    }
    switch r {
    case '+', '-', '.':
        return qStart // после знака должна идти непустая группа \w+
    case '@':
        return qAfterAt
    default:
        return qDead
    }

case qDead:
    return qDead

case qAfterAt:
    if w {
        return qDomain
    }
    return qDead

case qDomain:
    if w {
        return qDomain
    }
    switch r {
    case '-':
        return qAfterAt
    case '.':
        return qAfterDot
    default:
        return qDead
    }

case qAfterDot:
    if w {
        return qEmailEnd
    }
    return qDead

case qEmailEnd:
    if w {
        return qEmailWord
    }
    if s {
        return qSpaces
    }
    switch r {
    case '-':
        return qAfterDot
    case '.':
        return qAfterFinalDot
    case ';', ',':
        return qAfterSep
    default:
        return qDead
    }

case qEmailWord:
    if w {
        return qEmailWord
    }
    if s {
        return qSpaces
    }
    switch r {
    case '+':
        return qStart
    case '-':
        return qAfterHyphen
    case '.':
        return qAfterFinalDot
    case '@':
        return qAfterAt
    case ';', ',':
        return qAfterSep
    default:
```

```
            return qDead
        }

    case qSpaces:
        if w {
            return qLocal
        }
        if s {
            return qSpaces
        }
        if r == '.' || r == ';' || r == ',' {
            return qAfterSep
        }
        return qDead

    case qAfterFinalDot:
        if w {
            return qEmailWord
        }
        if s {
            return qAfterSep
        }
        return qDead

    case qAfterSep:
        if w {
            return qLocal
        }
        if s {
            return qAfterSep
        }
        return qDead

    case qAfterHyphen:
        if w {
            return qEmailWord
        }
        return qDead

    default:
        return qDead
    }
}

// isFinal проверяет, является ли состояние заключительным (q6–q10).
func isFinal(state int) bool {
    switch state {
    case qEmailEnd, qEmailWord, qSpaces, qAfterFinalDot, qAfterSep:
        return true
    default:
        return false
    }
}

// Run обрабатывает всю цепочку и возвращает последовательность
// пройденных состояний (трассу) и результат распознавания.
func Run(input string) ([]int, bool) {
    state := qStart
    trace := []int{state}
    for _, r := range input {
        state = nextState(state, r)
        trace = append(trace, state)
        if state == qDead {
            break // из q⊥ выхода нет, дальше читать бессмысленно
        }
    }
    return trace, isFinal(state)
}

// Accepts возвращает true, если цепочка принадлежит языку.
func Accepts(input string) bool {
    _, ok := Run(input)
    return ok
}
```

```
func main() {
    scanner := bufio.NewScanner(os.Stdin)
    // Увеличиваем стандартный лимит длины строки для длинных списков адресов.
    scanner.Buffer(make([]byte, 64*1024), 1024*1024)

    fmt.Println("Проверка строк конечным автоматом. Для выхода введите exit.")
    for {
        fmt.Print("> ")
        if !scanner.Scan() {
            break
        }

        input := scanner.Text()
        if input == "exit" {
            break
        }

        trace, ok := Run(input)
        fmt.Print("Трасса: ")
        for i, q := range trace {
            if i > 0 {
                fmt.Print(" → ")
            }
            fmt.Print(stateNames[q])
        }
        fmt.Println()
        if ok {
            fmt.Println("Цепочка принадлежит языку.")
        } else {
            fmt.Println("Цепочка не принадлежит языку.")
        }
    }

    if err := scanner.Err(); err != nil {
        fmt.Fprintf(os.Stderr, "Ошибка чтения ввода: %v\n", err)
        os.Exit(1)
    }
}
```

A.1 Тест EmailListDfa_test.go

```
package main

import (
    "regexp"
    "testing"
)

// Исходный паттерн. В Go (RE2) класс \w — только ASCII, поэтому
// сравнение ведётся на ASCII-алфавите: a, пробел, + - . @ ; ,
var pattern = regexp.MustCompile(`^((\w+([-+.]\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*)\s*[;,.]
{0,1}\s*)+$`)

// TestAgainstRegex перебирает ВСЕ цепочки длины 0…7 над алфавитом
// из 8 представителей классов и сравнивает ответ автомата с регулярным выражением.
func TestAgainstRegex(t *testing.T) {
    alpha := []rune("a +-.@;,")
    var buf []rune
    count := 0
    var rec func(depth int)
    rec = func(depth int) {
        s := string(buf)
        count++
        if Accepts(s) != pattern.MatchString(s) {
            t.Fatalf("расхождение на %q: автомат=%v, regex=%v", s, Accepts(s),
pattern.MatchString(s))
        }
        if depth == 7 {
            return
        }
```

```
        for _, r := range alpha {
            buf = append(buf, r)
            rec(depth + 1)
            buf = buf[:len(buf)-1]
        }
    }
    rec(0)
    t.Logf("проверено цепочек: %d", count)
}
```